

Deep-Dive Explanation — wrist_to_base.py

Purpose & Data Flow

The node detects up to two hands using MediaPipe, finds the wrist landmark (index 0) in the color image, samples a robust depth at that pixel from the aligned depth-to-color image, deprojects the pixel to 3D in the color optical frame, transforms it via TF2 into a base/world frame, and publishes both PointStamped messages (left/right) and RViz markers. It also publishes a per-frame annotated debug image for visual verification.

Parameters & QoS

Parameters (CLI-overridable): color_topic, depth_topic, info_topic, source_frame, target_frame, min_det_conf, min_trk_conf. Defaults assume a RealSense-style layout and a base frame named platform_base.

Subscriptions use QoSProfileProfiles.SENSOR_DATA to match camera drivers; publishers use queue depth 10.

Subscriptions & Publishers

Subscribed:

- Color image (BGR8) — color_topic
- Aligned depth (uint16 mm or float32 m) — depth_topic
- Color CameraInfo — info_topic

CameraInfo initializes a PinholeCameraModel to access fx, fy, cx, cy.

Published:

- /wrist_left_point_base, /wrist_right_point_base — geometry_msgs/PointStamped in target_frame
- /wrist_marker — visualization_msgs/Marker
- /wrist_marker_array — visualization_msgs/MarkerArray
- /wrist/debug_image — sensor_msgs/Image annotated

Robust Depth Sampling

robust_depth_at(u, v, depth, h, w) expands search windows $7 \rightarrow 11 \rightarrow 15$ and returns the median of non-zero pixels (in meters) for stability. If no valid pixels are found it returns NaN. This mitigates typical hand-region holes/noise in depth sensors.

Deprojection to Camera 3D

With f_x , f_y , c_x , c_y from `CameraInfo`, compute:

$$X_c = (u - c_x) * z / f_x$$

$$Y_c = (v - c_y) * z / f_y$$

$$Z_c = z$$

Result is the wrist point in `source_frame` (e.g., `cam2_color_optical_frame`).

TF2 Transform to Base

Wrap (X_c , Y_c , Z_c) in a `PointStamped` stamped in `source_frame`. Use `lookup_transform(target_frame, source_frame, Time())` to get the latest transform, then `tf2_geometry_msgs.do_transform_point` to obtain the point in the base frame. Publish to left/right topics based on `MediaPipe` handedness. If TF is missing, warn once and skip publishing for that frame.

Debug Overlay & Markers

Draw `MediaPipe` hand skeleton; mark the wrist pixel and overlay label and z (meters) on the debug image. Create a persistent sphere `Marker` (≈ 0.12 m diameter) per hand (e.g., red for Left, green for Right) and also publish a `MarkerArray`.

Alignment & Depth Dtype Checks

Depth image is accepted in `uint16` (interpreted as millimeters and divided by 1000) or `float32` (meters). Unexpected dtypes produce a warning and are ignored. If `depth.header.frame_id != source_frame`, warn once to enable depth-to-color alignment or adjust `source_frame`.

Logging & Throttling

Informational logs include pixel, depth, camera-frame coordinates, and base-frame coordinates. Logging is throttled to avoid excessive spam during streaming.

Quick Tips

- Ensure depth is aligned to color (e.g., `RealSense align_depth:=true`).
- Verify a TF chain exists from `target_frame` to `source_frame` (URDF + `robot_state_publisher` or static TF).
- If `CameraInfo` is missing, check you are using the matching `.../camera_info` for the color stream.
- For noisy depth, increase illumination or allow the larger window sizes to kick in.